

MaintainIQ

Technical Documentation

AI-Powered QR Maintenance & Asset History Platform

Final Hackathon MERN— Advanced Full-Stack + GenAI)

Version 2.0 — Post-Submission RBAC/OTP/Cookie-Auth Upgrade

Table of Contents

1. System Overview
2. Architecture — Tech Stack, Authentication Model
3. Role-Based Access Control — Roles, Approval Workflow
4. OTP Verification System — Signup OTP, Guest Report OTP
5. Data Models — User, Asset, Issue, Supporting Collections
6. API Reference
7. Key Business Rules
8. Admin Dashboard
9. Deployment Notes
10. Known Limitations & Future Work

1. System Overview

MaintainIQ gives every physical asset (projector, AC, camera, etc.) a digital identity: a unique asset code, a QR-accessible public page, an AI-assisted issue reporting flow, a controlled maintenance workflow, and a permanent, exportable service history.

The platform serves four distinct audiences through role-specific dashboards: a Super Admin who governs the whole system, Admins who manage day-to-day operations, Technicians who action assigned work, and both registered Users and anonymous Guests who report issues via QR scan.

This document covers the architecture as it stands after a substantial post-submission upgrade: the original CRUD prototype was rebuilt with a full role-based access control system, dual OTP email verification, cookie-based session authentication, and a redesigned six-tab admin experience.

2. Architecture

2.1 Tech Stack

Layer	Technology
Frontend	React (Vite), Tailwind CSS, React Router
Backend	Node.js, Express
Database	MongoDB + Mongoose
Auth	JWT via httpOnly cookies, bcrypt, custom role middleware
Email / OTP	Twilio SendGrid (Single Sender Verification)
AI	OpenRouter multi-model fallback chain (Issue Triage)
QR Codes	qrcode (npm)
PDF Generation	jsPDF — asset posters and issue history reports
Media Storage	Cloudinary (technician certifications, asset photos)
Deployment	Vercel (frontend) + Render (backend)

2.2 Authentication Model

Sessions are managed via a signed JWT stored in an httpOnly cookie named "token" — never in localStorage or a JS-readable location. The cookie's security attributes are environment-aware:

- Production: secure: true, sameSite: 'none' — required because the frontend (Vercel) and backend (Render) are on different HTTPS domains.
- Local development: secure: false, sameSite: 'lax' — required because secure cookies are refused by browsers over plain HTTP.

The switch is automatic, driven by NODE_ENV, which Render sets in production and which is absent locally — no manual toggling required.

CORS is locked to an exact origin (process.env.FRONTEND_URL) with credentials: true; a wildcard origin is incompatible with credentialed cookie requests.

3. Role-Based Access Control

3.1 Roles

Role	Created via	Approval required?	Dashboard
superadmin	Seeded once at server startup from env vars	No — pre-approved	Admin Dashboard (all 6 tabs)
admin	Signup form	Yes — Super Admin approves	Admin Dashboard (5 tabs, no Administrators tab)
technician	Signup form + certification upload	Yes — Admin or Super Admin approves	Technician Dashboard
user	Signup form	No — auto-approved after email OTP	User Dashboard

3.2 Approval Workflow

The User schema tracks two independent states: `isVerified` (email OTP completed) and `status` (pending / approved / revoked, the admin-approval gate). A technician or admin account must satisfy both before login succeeds; a user account only needs `isVerified`.

Approving, rejecting, and later revoking or re-granting all funnel through one endpoint, `PUT /auth/:id/status` — the Approvals tab and the Management tables (Administrators / Technicians / Users) both just flip this field. A superadmin bypass is built into the role middleware itself: `allowRoles(...)` always lets a superadmin through, regardless of which roles are explicitly listed, so routes only need to name ‘admin’ and superadmin automatically inherits access.

Deleting an account is blocked server-side unless its status is already ‘revoked’ — this is enforced in the controller, not just via a disabled button client-side.

4. OTP Verification System

Two independent OTP flows share one reusable two-step frontend component, `OtpModal.jsx`: an email-collection step (skipped if the email is already known) followed by a code-entry step with a live countdown and a Resend action that re-enables only once the timer expires.

4.1 Signup OTP

- Triggered by `POST /auth/register`; a 6-digit numeric code is emailed and stored with a 5-minute expiry.
- `POST /auth/verify-otp` checks the code and expiry, then either logs the user straight in via cookie (auto-approved ‘user’ role) or leaves them pending in the Approvals queue.
- `POST /auth/resend-otp` regenerates the code without requiring the person to resubmit the whole form.

4.2 Guest Report OTP

- A separate `GuestOtp` collection (independent of the User model — no account is created) backs `POST /public/otp/send` and `/otp/verify`.
- The code expires after 2 minutes. On success, the server issues a short-lived signed JWT (purpose: ‘guest-report’, 10-minute expiry) which the frontend must attach when calling `POST /issues`.
- An optional `Auth` middleware on the issue-creation route trusts a valid session cookie if present (logged-in users bypass OTP entirely); otherwise it requires and verifies the guest report token, matching its embedded email against the submitted `reporterInfo.email`.

5. Data Models

5.1 User

Field	Type	Notes
role	enum	user admin technician superadmin
status	enum	pending approved revoked
isVerified	Boolean	Email OTP completed
specialty	String	Technician only
profilePic	String	Cloudinary URL of certification/evidence doc
approvedBy	String	Name of the admin/superadmin who last actioned this account
otp / otpExpiry	String / Number	Signup OTP state

5.2 Asset

Field	Type	Notes
assetCode	String, unique	Human-readable identifier
category	String	Drawn from the shared category list, or a custom "Other" value
status	enum	Operational Issue Reported Under Inspection Under Maintenance Out of Service Retired
qrUrl	String	Base64 QR code generated at creation time, points to the public asset page
imageUrl	String	Cloudinary photo of the physical asset (optional)
assignedTechnician	ObjectId ref User	Optional

5.3 Issue

Field	Type	Notes
issueNumber	String, unique	Collision-safe; timestamp + random suffix
status	enum	Reported → Assigned → Inspection Started → Maintenance In Progress → Resolved → Closed/Reopened
reporterInfo	Object	name, email, phone — populated for both guest and logged-in reporters
assignedTechnician / assignedBy	ObjectId ref User / String	Who is assigned, and who assigned them
aiSuggested / aiEdited	Boolean	Whether the AI Triage suggestion was used and/or edited before submission

5.4 Supporting Collections

- MaintenanceRecord — technician notes, cost, time spent, final condition; required before an issue can be marked Resolved.
- History — a running audit log (actor, action, timestamp) tied to an asset and, optionally, a specific issue via relatedIssueId; powers both the asset history endpoint and the per-issue timeline in the Issue History Report.
- GuestOtp — one document per email, overwritten on each resend; not linked to any User account.

6. API Reference

Method	Endpoint	Access
POST	/api/auth/register	Public — starts signup OTP flow
POST	/api/auth/verify-otp	Public
POST	/api/auth/resend-otp	Public
POST	/api/auth/login	Public
POST	/api/auth/logout	Logged in
GET	/api/auth/me	Logged in
GET	/api/auth/technicians	Logged in — approved technicians only
GET	/api/auth/pending	Admin, Super Admin
GET	/api/auth/list	Admin, Super Admin
PUT	/api/auth/:id/status	Admin, Super Admin
DELETE	/api/auth/:id	Admin, Super Admin (admin targets: Super Admin only)
POST	/api/assets	Admin, Super Admin — multipart, optional image
GET	/api/assets	Logged in
PUT	/api/assets/:id	Admin, Super Admin — multipart, optional image
DELETE	/api/assets/:id	Super Admin only
GET	/api/public/assets	Public
GET	/api/public/asset/:code	Public
POST	/api/public/otp/send	Public — guest reporting
POST	/api/public/otp/verify	Public — guest reporting
POST	/api/issues	Public (OTP-gated) / Logged in (bypass)
GET	/api/issues	Logged in — supports reporterEmail, technician, status, category filters
PUT	/api/issues/:id/assign	Admin, Super Admin
PUT	/api/issues/:id/status	Admin, Technician
POST	/api/maintenance	Admin, Technician
GET	/api/maintenance/issue/:issueId	Logged in
GET	/api/history/:assetId	Logged in
GET	/api/history/issue/:issueId	Logged in
POST	/api/ai/triage	Public
POST	/api/upload	Logged in — Cloudinary evidence upload

7. Key Business Rules

- Duplicate asset codes are rejected at creation.
- Issue status transitions follow a fixed state machine — no skipping steps.
- An issue cannot be marked Resolved without at least one maintenance note on file.
- Maintenance cost cannot be negative.
- A Closed issue is locked from further edits until explicitly reopened.
- Only a superadmin may approve, revoke, or delete another admin account.
- An account can only be deleted once its status is 'revoked' — enforced both in the UI (disabled button) and the API (rejected otherwise).
- Only a superadmin can delete an asset; editing is available to both admin and superadmin.
- Deleting an asset never deletes its issue history — related issues persist and are labeled 'Asset Removed' wherever the asset would normally be referenced (Issue Management, Technician dashboard, User dashboard), and are excluded from the Overview tab's live Open/Critical issue counts since they are no longer actionable.
- The Assign Technician control and the (legacy poster) download are both disabled for issues whose asset has been removed, since there is no physical asset left to act on — but the full Issue History Report remains downloadable regardless, since it exists to preserve history.
- Category values across the entire app (asset creation, issue reporting, every filter) are drawn from one shared list plus an 'Other' custom-entry escape hatch — preventing typo-driven fragmentation such as 'Electronics' / 'Electronics45' / 'Electroinics SA' being treated as different categories.

8. Admin Dashboard

The dashboard header is role-aware, reading "MAINTAINIQ / SUPER ADMIN" or "MAINTAINIQ / ADMINISTRATOR" depending on who is logged in. Six tabs are available (five for a plain Admin, who does not see Administrators):

Tab	Purpose
Overview	Asset registry (card/table, filters, sort, search), asset creation, edit, delete
Approvals	Pending technician/admin signups; certification documents viewable inline
Issue Management	Card/table, filters, sort, technician assignment, Issue History Report export
Administrators	Super Admin only — Grant/Revoke, gated Delete
Technicians	Admin + Super Admin — Grant/Revoke, gated Delete, specialty column
Users	Admin + Super Admin — Grant/Revoke, gated Delete

The Issue History Report (Issue Management → Report column) is a multi-section PDF built at download time: it fetches the issue's maintenance records and audit-log timeline live, then renders Issue Details (with the asset's photo shown top-right, fixed-size and top-aligned), Asset snapshot (or an "Asset Removed" notice), Reporter info, Assignment trail, a chronological Timeline, and every Maintenance Record, paginating automatically for long histories.

9. Deployment Notes

- Vercel per-deployment preview URLs are frozen at build time — only the assigned Production Domain reliably reflects the latest push.
- SPA client-side routing on Vercel requires a vercel.json rewrite (catch-all to index.html) or deep links 404 on refresh.
- Gmail SMTP from a cloud host (Render, Railway, Heroku, etc.) is unreliable in production: shared outbound IP ranges get throttled by Gmail, producing intermittent ETIMEDOUT errors that do not reproduce locally. SendGrid (Single Sender Verification — no owned domain required) replaced Gmail SMTP for this reason.
- When sending FormData via axios, never manually set the Content-Type header — the browser must generate it (it embeds a random multipart boundary). Overriding it causes multer/busboy to hang indefinitely with no thrown error, which presents identically to a frontend bug (a stuck loading state).
- Cookie secure/sameSite settings must differ between local HTTP development and production HTTPS cross-site deployment — hardcoding secure: true breaks local testing entirely. Solved via a NODE_ENV check.
- QR codes are generated once, at asset-creation time, using the FRONTEND_URL environment variable in effect at that moment — changing the variable later does not retroactively regenerate existing QR codes.

10. Known Limitations & Future Work

- No automated test suite.
- The Issue History Report is Admin-only; Technicians and Users cannot generate it for issues they are party to.
- "My Reported Issues" (User Dashboard) has no search or filtering yet.
- Asset deletion is permanent — no soft-delete, restore, or bulk-import/export tooling.
- The "My Reported Issues" tab fetches each issue's latest maintenance note via a separate request per issue; acceptable at hackathon scale but would need a backend join for larger data sets.